

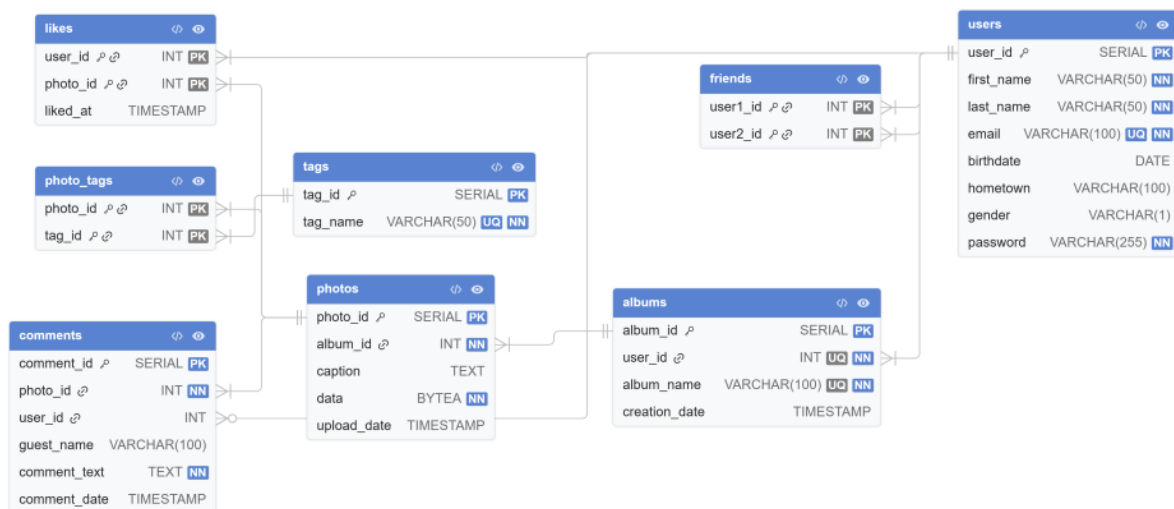
ΣΧΒΔ – Εργασία #3

Νίκος Ρούσκας – 1115202400177

Θανάσης Παντίσκας – 1115202400151

Setup

- Προϋπόθεση για να τρέξει η εφαρμογή αποτελεί να έχει κάνει ο χρήστης download την python3 καθώς και την PostgreSQL.
- Η βάση δημιουργείται από τον postgres superuser και το main πρόγραμμα app.py έχει πρόσβαση στην βάση της εφαρμογής ως postgres superuser. Έτσι στο setup.sh υπάρχει η εντολή να ζητείται από τον χρήστη ο κωδικός του postgres superuser για να μπορεί να δημιουργηθεί η βάση και γίνεται export για να συνδεθεί η εφαρμογή σε αυτήν ως χρήστης postgres.
- Επίσης στο setup δημιουργείται ένα python virtual environment όπου γίνεται install το pip και τα requirement που είναι οι βιβλιοθήκες Flask, psycopg2-binary.
- Έπειτα δημιουργείται η βάση και το schema σε αυτή και μετά τρέχει η εφαρμογή app.py



Application

Triggers: Constraints που απαιτούν αναφορά σε άλλους πίνακες

- **trg_no_self_comment**: Εκτελείται BEFORE INSERT στον πίνακα comments και αποτρέπει έναν χρήστη από το να σχολιάσει τη δική του φωτογραφία.

- **trg_no_self_like:** Αντίστοιχος έλεγχος BEFORE INSERT στον πίνακα likes, που αποτρέπει έναν χρήστη από το να κάνει like στη δική του φωτογραφία.
- **trg_normalize_friends:** Εκτελείται BEFORE INSERT στον πίνακα friends και εξυπηρετεί δύο σκοπούς: πρώτον, αποτρέπει την αυτοφιλία ($user1_id = user2_id$), και δεύτερον, κανονικοποιεί αυτόματα τη σειρά των IDs ώστε πάντα $user1_id < user2_id$, σύμφωνα με το constraint του σχήματος. Έτσι ο application κώδικας δεν χρειάζεται να φροντίζει για την σειρά εισαγωγής.

1. Διαχείριση Χρηστών

- Για την εγγραφή χρήστη (1α), αρχικά ελέγχουμε αν υπάρχει ήδη στη βάση χρήστης με το ίδιο email. Σε περίπτωση που υπάρχει, εμφανίζουμε μήνυμα σφάλματος. Αλλιώς εισάγουμε το νέο χρήστη. Η ημερομηνία γέννησης (birthdate) είναι προαιρετική — αν δεν δοθεί, αποθηκεύεται ως NULL. Επίσης Για την είσοδο χρήστη και όχι απλά guest, ελέγχουμε αν το email υπάρχει στη βάση και αν ο κωδικός ταιριάζει. Αν η σύνδεση είναι επιτυχής, αποθηκεύουμε το user_id στο session ώστε να γνωρίζουμε ποιος είναι συνδεδεμένος σε κάθε επόμενο request. Αν ο χρήστης είναι ήδη συνδεδεμένος ανακατευθύνεται απευθείας στην αρχική σελίδα.
- Για την προβολή και αναζήτηση φίλων (1β), η διαδρομή /friends εμφανίζει τη λίστα φίλων του συνδεδεμένου χρήστη. Επειδή ο πίνακας friends αποθηκεύει κάθε φιλία μία φορά με $user1_id < user2_id$, το ερώτημα χρησιμοποιεί UNION για να βρει τον χρήστη και στις δύο στήλες. Η αναζήτηση γίνεται με LIKE σε όνομα, επώνυμο και email. Η προσθήκη φίλου γίνεται με INSERT κανονικοποιώντας τη σειρά των IDs (min/max), και ON CONFLICT DO NOTHING για τα duplicates.
- Για τη δραστηριότητα χρηστών (1γ), η βαθμολογία συνεισφοράς υπολογίζεται ως άθροισμα φωτογραφιών και σχολίων σε φωτογραφίες άλλων χρηστών. Χρησιμοποιούμε δύο υποερωτήματα με LEFT JOIN και COALESCE για να χειριστούμε χρήστες με μηδενική δραστηριότητα, και LIMIT 10 για τους κορυφαίους.

2. Διαχείριση Άλμπουμ και Φωτογραφιών

- Για την περιήγηση στα άλμπουμ και τις φωτογραφίες υλοποιήθηκαν σελίδες προβολής όλων των άλμπουμ, προβολής ενός συγκεκριμένου άλμπουμ και προβολής μίας συγκεκριμένης φωτογραφίας. Σύμφωνα με την εκφώνηση, τα άλμπουμ και οι φωτογραφίες θεωρούνται δημόσια, άρα μπορούν να προβληθούν τόσο από εγγεγραμμένους όσο και από μη εγγεγραμμένους χρήστες. Οι εικόνες αποθηκεύονται στη βάση στο πεδίο

data ως δυαδικά δεδομένα (BYTEA) και, για την εμφάνισή τους στα HTML templates, μετατρέπονται από το Flask σε base64 string ώστε να χρησιμοποιηθούν ως src σε HTML img element.

- Η δημιουργία άλμπουμ επιτρέπεται μόνο σε συνδεδεμένους χρήστες. Κατά τη δημιουργία, το νέο άλμπουμ συνδέεται με το user_id του χρήστη που είναι αποθηκευμένο στο session. Αν μη συνδεδεμένος χρήστης προσπαθήσει να δημιουργήσει άλμπουμ ή να ανεβάσει φωτογραφία, ανακατευθύνεται στη σελίδα σύνδεσης. Η μεταφόρτωση φωτογραφιών γίνεται μέσα από συγκεκριμένο άλμπουμ και πριν επιτραπεί ελέγχεται ότι το άλμπουμ ανήκει στον συνδεδεμένο χρήστη.
- Κατά τη μεταφόρτωση φωτογραφίας, ο χρήστης μπορεί να εισάγει και tags. Τα tags αποθηκεύονται στον πίνακα tags, ενώ η σχέση φωτογραφίας-tag αποθηκεύεται στον ενδιάμεσο πίνακα photo_tags. Χρησιμοποιείται ON CONFLICT DO NOTHING, ώστε να αποφεύγονται διπλότυπα tags και διπλότυπες συσχετίσεις φωτογραφίας-tag.
- Υλοποιήθηκε επίσης διαγραφή φωτογραφιών και άλμπουμ. Οι επιλογές διαγραφής εμφανίζονται μόνο όταν ο συνδεδεμένος χρήστης είναι ο ιδιοκτήτης του αντίστοιχου άλμπουμ ή της φωτογραφίας. Παράλληλα, ο έλεγχος ιδιοκτησίας γίνεται και στο backend πριν εκτελεστεί η διαγραφή, ώστε να μην είναι δυνατή η μη εξουσιοδοτημένη διαγραφή μέσω χειροκίνητου POST request. Στη βάση χρησιμοποιείται ON DELETE CASCADE, ώστε με τη διαγραφή άλμπουμ ή φωτογραφίας να διαγράφονται και οι σχετικές εγγραφές.

3. Διαχείριση Ετικετών

- (3a/3b), για να προβάλλετε τις φωτογραφίες ανά ετικέτα, χρησιμοποιήσαμε έναν διακόπτη "All / Mine" μέσω ενός query parameter filter=mine. Εάν ο χρήστης δεν είναι συνδεδεμένος και ζητήσει "Mine", ανακατευθύνεται στη σελίδα σύνδεσης.
- (3c) Για να εμφανίσουμε τις πιο δημοφιλείς ετικέτες, το ερώτημα μετράει πόσες φωτογραφίες έχει κάθε tag χρησιμοποιώντας COUNT και GROUP BY, και τις ταξινομεί φθίνουσα.
- (3d) για να αναζητήσετε φωτογραφίες, ο χρήστης εισάγει λέξεις χωρισμένες με κενό. Το ερώτημα χρησιμοποιεί WHERE tag_name = ANY(...) για να βρει φωτογραφίες που έχουν έστω ένα από τα tags, και HAVING COUNT(DISTINCT tag_id) = N για να κρατήσει μόνο αυτές που έχουν όλα τα tags.

4. Σχόλια και Likes

- Για κάθε φωτογραφία υλοποιήθηκε ξεχωριστή σελίδα προβολής, στην οποία εμφανίζονται η φωτογραφία, η λεζάντα της, ο αριθμός των likes και οι διαθέσιμες ενέργειες του χρήστη.
- Στη λειτουργία σχολίων, τόσο οι συνδεδεμένοι χρήστες όσο και οι επισκέπτες μπορούν να αφήσουν σχόλιο σε μία φωτογραφία. Για τους συνδεδεμένους χρήστες αποθηκεύεται το `user_id`, ενώ για τους επισκέπτες αποθηκεύεται ένα απλό `guest_name`. Για αυτόν τον λόγο, στον πίνακα `comments` το `user_id` επιτρέπεται να είναι `NULL`, ενώ προτέθηκε και πεδίο `guest_name`. Έτσι κάθε σχόλιο μπορεί να ανήκει είτε σε εγγεγραμμένο χρήστη είτε σε επισκέπτη.
- Για τους εγγεγραμμένους χρήστες εφαρμόζεται ο περιορισμός ότι δεν μπορούν να σχολιάσουν δικές τους φωτογραφίες. Ο ιδιοκτήτης της φωτογραφίας εντοπίζεται μέσω της σχέσης `photos -> albums -> users`, αφού η φωτογραφία ανήκει σε άλμπουμ και το άλμπουμ έχει ιδιοκτήτη. Αν το `user_id` του σχολιαστή είναι ίδιο με το `user_id` του ιδιοκτήτη του άλμπουμ, η εισαγωγή σχολίου απορρίπτεται και εμφανίζεται μήνυμα λάθους.
- Για την λειτουργία like, μόνο οι συνδεδεμένοι χρήστες μπορούν να κάνουν like σε φωτογραφίες. Το ζεύγος (`user_id`, `photo_id`) χρησιμοποιείται ως μοναδικός συνδυασμός, ώστε ένας χρήστης να μην μπορεί να κάνει πολλαπλά likes στην ίδια φωτογραφία. Στη σελίδα κάθε φωτογραφίας εμφανίζεται ο συνολικός αριθμός likes, ενώ ο αριθμός λειτουργεί ως σύνδεσμος προς ξεχωριστή σελίδα που εμφανίζει τα ονόματα των χρηστών που έκαναν like.
- Τέλος, υλοποιήθηκε αναζήτηση σχολίων. Ο χρήστης μπορεί να εισάγει κείμενο, και το σύστημα αναζητά σχόλια που ταιριάζουν ακριβώς με αυτό το κείμενο, αγνοώντας διαφορές πεζών/κεφαλαίων. Τα αποτελέσματα εμφανίζουν τον χρήστη που έκανε το σχόλιο, το κείμενο του σχολίου, την ημερομηνία και τον σύνδεσμο προς τη φωτογραφία στην οποία έγινε το σχόλιο.